AI 에이전트 클라우드 실습 워크북 — minikube 위에 온프레미스 에이전트 올리기

온프레미스 에이전트를 안전하게 만들고, 재서 고르는 프레임워크

8주차 · 환경: Windows 11 · WSL2 Ubuntu · Docker · minikube(docker 드라이버) · RTX 5070 Laptop 8GB

형식: 장마다 완료 목표 → 그림 → 실행(명령 상자 · 행별 해설) → 관찰 → 장애·복구 → 정리 → 성공 확인 — 그리고 잰다

저장소: github.com/moongu0891-ship-it/onprem-agent · 커밋 a35249c · 2026-10-12

차례

00. 실습 지도와 사용 방법

01. GPU 를 클러스터에 넘길 수 있는가 — 이 판정이 뒤 장의 모양을 정한다

02. 클러스터 둘러보기 — 노드 · 네임스페이스 · 첫 Pod

03. 에이전트 이미지 만들기 — 코드를 상자 하나에

04. 설정과 비밀을 밖으로 — ConfigMap · Secret · 상담 화면 Pod 하나

05. 나눠 주기와 무상태 — 상담 화면 ×3, 로드 밸런서, Pod 를 죽여 보기

06. 자동으로 늘리고 줄이기 — metrics-server · HPA · 부하

07. 모델 서버를 클러스터 안으로 — GPU Pod · PVC · 상담 화면을 실제 모델로

08. 엣지 — https · 되돌림 · 방화벽 · CDN 흉내

09. 관문 — 로그인 · 요청 제한 · LiteLLM 게이트웨이

10. 데이터 — PostgreSQL + pgvector 하나로, 무상태 되찾기

11. 관측 — /metrics · Prometheus · Grafana

12. CI/CD — Jenkins 를 클러스터에, 노트북 GPU 를 일꾼으로

13. IaC — make cloud-up 한 줄, 처음부터 다시 올리는 시간

마치며 — 이 워크북에서 잰 것

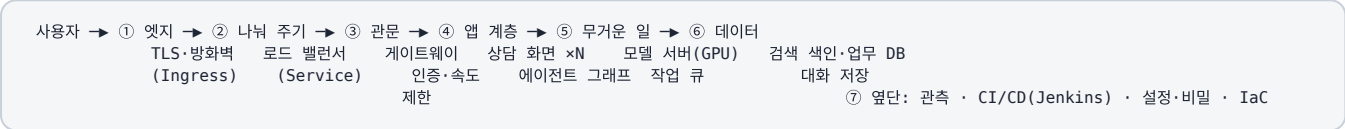
부록 — 스크립트 한 줄씩

7주차까지 만든 에이전트(상담 화면 · 에이전트 그래프 · 업무 기능(도구) 서버 · 모델 서버 · 게이트웨이)를 **실제 클라우드 서비스의 층**대로 minikube 위에 올린다. 형식은 'Minikube에서 AWS EKS까지' 워크북과 같다 — 장마다 **완료 목표 → 그림 → 실행(명령 한 줄씩 해설) → 관찰 → 장애·복구 → 정리 → 성공 확인**. 그리고 이 저장소답게 장마다 **잰다**.

- 환경: Windows 11 · WSL2 Ubuntu · Docker · minikube(docker 드라이버) · RTX 5070 Laptop 8GB · WSL2 메모리 15GB
- 상용 클라우드 없이 노트북 한 대로. 진짜로 못 만드는 칸(전 세계 CDN, 노드 자동 확장)은 흉내와 설명으로 채우고 그렇게 적는다.
- 명령은 모두 `~/onprem-agent` 에서 실행한다. 결과는 `results/cloud/` 에 남고(Git 제외), 확정본은 `reports/cloud_*.md` 로 옮긴다.

00. 실습 지도와 사용 방법

완료 목표 · 어느 층을 어느 장에서 채우는지, 노트북 메모리를 어떻게 나눠 쓰는지 이해한다.



층 (그림의 칸)	실제 클라우드에서는	이 워크북에서 만드는 것	장	잰다
화면	브라우저가 보는 프런트엔드	7주차 상담 화면(`/`)·관리자 화면(`/admin`)을 Pod 로	03·04	이미지 크기·빌드 시간, 화면이 뜨기까지
① 엣지 · TLS·방화벽	ACM · WAF	Ingress-NGINX + 자체 서명 인증서(https), NetworkPolicy 로 모델 서버는 앱만 접근	08	http→https 되돌림, 막힌 통신 확인
① CDN	CloudFront	**없음(흉내만)** — 정적 파일에 캐시 머리말(Cache-Control) 붙이고 두 번째 요청이 304 인지 본다	08	304 비율
② 로드 밸런서	ALB · NLB	Service(ClusterIP) 가 상담 화면 Pod N 개에 나눠 준다 — 어느 Pod 가 받았는지 답 머리말로 본다	05	Pod 별 요청 수
② 오토스케일링	HPA · Auto Scaling	HPA 로 상담 화면 Pod 1 → 4 (CPU 기준) · 노드 자동 확장은 **없음(설명)**	06	부하 중 Pod 수와 p95
③ API 게이트웨이 · 인증 · 속도 제한	API Gateway · Cognito · rate limit	Ingress 에 기본 인증(Basic Auth) + 분당 요청 제한 · LiteLLM 게이트웨이가 모델 앞	09	429 가 나오는 요청률
④ 앱 서버(무상태) ×N	ECS · Cloud Run	상담 화면 Pod 는 **무상태** — 대화·승인 대기는 ⑥ 의 DB 에	05	Pod 를 죽여도 승인 대기가 이어지나
④ 컨테이너 오케스트레이터	Kubernetes · ECS	minikube 자체	02	—
⑤ 모델 서빙(GPU)	SageMaker · Triton	SGLang Qwen3-4B(FP8) — **GPU 가 Pod 에 보이면 클러스터 안, 아니면 호스트 Docker 에 두고 클러스터에서 붙음**(01장이 정함)	01·07	첫 글자 시간·처리량(3주차 측정기 그대로)
⑤ 작업 큐 · 워커	SQS · Celery	전체 시험(1시간)을 Job 으로 — 큐는 Jenkins 가 맡는다	12	—

층 (그림의 칸)	실제 클라우드에서는	이 워크북에서 만드는 것	장	잰다
⑤ 배치 파이프라인	Airflow · Batch	검색 색인 만들기를 Job 으로(앱 시작 전에 한 번)	10	색인 시간
⑥ 벡터 DB · 관계형 DB	pgvector · RDS	PostgreSQL + pgvector StatefulSet 하나로 둘 다(2주차 어댑터 그대로) · 대화·승인 대기 저장도 여기	10	Pod 재시작 뒤 데이터가 남나
⑥ 객체 저장소 · 캐시	S3 · Redis	PVC 에 모델·색인 파일 · 캐시는 모델 서버의 접두사 캐시(3주차) — Redis 는 **없음(설명)**	10	—
⑦ 관측	CloudWatch · Datadog	Prometheus + Grafana(가벼운 구성) + Langfuse(에이전트 추적)	11	대시보드에 p95·승인 수
⑦ CI/CD	GitHub Actions · ArgoCD	**Jenkins** 를 minikube 에, 노트북(GPU)을 일꾼으로 — 올릴 때 단위·모듈, 매일 밤 전체 시험	12	밤 시험이 돌고 빨간 불 보고서가 남나
⑦ 설정 · 비밀	Secrets Manager	ConfigMap(창구 정의 `desk.yaml`) · Secret(게이트웨이 키, DB 비밀번호)	04	—
⑦ IaC	Terraform	`deploy/` 의 매니페스트·Helm 값 파일이 그 자체. `make cloud-up` 한 줄로 전부	13	처음부터 다시 올리는 시간

메모리 예산 (WSL2 15GB): ~~minikube 노드 6GB + 모델 서버 4~5GB~~ → **07장에서 재서 고침:** 노드 12GB(`docker update`) 안에 모델 서버 7.9GB(안정, 첫 시작 땐 11GB 넘게) + 상담 화면 0.8GB + 제어부·DB·관측 2~3GB. 모델 서버가 뜰 때는 다른 것을 내려 둔다. 전체 시험 중 임베더 2GB 는 노드 밖(호스트). **전체 시험(12장)은 관측·Jenkins 외 실습 서비스를 내린 상태에서만** 돌린다 — Jenkins 가 시작 전에 확인한다. 장마다 '이 장에 필요한 것만' 띄우고 끝에 내린다.

장 순서와 의존: 00 → 01(GPU 판정) → 02 → 03 → 04 → 05 → … 순서대로. 01 의 결과가 07 의 모양을 정하므로 **01 을 먼저 노트북에서 돌리고 결과를 적은 뒤** 뒤 장을 쓴다(이 문서는 그 순서대로 자란다).

코드와 해설을 읽는 순서

각 코드 상자의 왼쪽 숫자는 해설을 찾는 행 번호다. 터미널에는 숫자를 빼고 명령만 넣는다. 상자 이름은 `장-번호`(예 `01-003`). 행 끝의 `\` 는 다음 줄까지 한 명령이다. 모든 명령은 `~/onprem-agent` 에서, WSL2 Ubuntu 터미널에서 실행한다. `kubectl` · `minikube` · `docker` 는 가상환경과 무관하지만, `python scripts/…` 와 `make …` 는 이 저장소의 가상환경에서 — 프롬프트가 `(.venv)` 가 아니면 `deactivate; source .venv/bin/activate`. 긴 스크립트는 `deploy/scripts/` 에 있고 부록에서 한 줄씩 읽는다.

표현	읽는 방법
`-n lab`	조작할 네임스페이스(작업 구역) 이름. 이 워크북은 전부 `lab`
`-p agent`	minikube 프로필(클러스터 이름). 이 워크북은 전부 `agent`
`kubectl get/describe/logs`	조회 / 상세와 사건 / 컨테이너 출력
`--overrides='{...}'`	명령 한 줄로 Pod 를 만들 때 자원 요구 같은 세부를 JSON 으로 덧붙임
성공 판정	종료 코드 0 + 각 장의 '성공 확인' 에 적은 출력

01. GPU 를 클러스터에 넘길 수 있는가 — 이 판정이 뒤 장의 모양을 정한다

완료 목표 · minikube 안의 Pod 가 `nvidia-smi` 로 노트북 GPU 를 보는지 확인하고, 결과를 적는다.

```
그림 01 | GPU 가 어디까지 보이나
  노트북 GPU ─▶ WSL2 (nvidia-smi 됨, 3주차 확인) ─▶ Docker (--gpus, 3주차 서빙 확인) ─▶ minikube 노드(컨테이너) ─▶ Pod   ← 여기가 물음표
```

minikube docker 드라이버는 노드 자체가 Docker 컨테이너다. 그 안에서 또 컨테이너(Pod)가 도는 구조라, GPU 가 두 겹을 통과해야 한다. minikube 는 `--gpus all` 옵션과 NVIDIA 장치 플러그인으로 이를 지원하지만 WSL2 에서는 되는 조합이 좁다. 그래서 **먼저 잰다**.

실행

Ubuntu Bash | 01-001

```
bash deploy/scripts/install-minikube.sh
minikube start -p agent --driver=docker --container-runtime=docker --gpus=all --memory=6g --cpus=4
kubectl create namespace lab
minikube -p agent addons enable nvidia-device-plugin
kubectl -n kube-system get pods | grep nvidia
```

1행 kubectl · minikube · helm 을 설치한다(이미 있으면 건너뜀). 스크립트 한 줄씩 읽기는 부록. 2행 클러스터를 만든다. `-p agent` : 프로필 이름. `--driver=docker` : 노드를 Docker 컨테이너로. `--gpus=all` : 노드 컨테이너에 GPU 를 넘긴다(docker 의 `--gpus all` 과 같음). `--memory=6g --cpus=4` : 노드 상한 — 15GB 중 모델 서버 몫을 남긴다. 3행 작업 구역 `lab` 을 만든다. 이 워크북의 모든 자원은 여기에. 4행 NVIDIA 장치 플러그인을 켠다 — 노드의 GPU 를 Kubernetes 가 `nvidia.com/gpu` 자원으로 세게 한다. 5행 플러그인 Pod 가 떴는지 본다. `Running` 이어야 한다.

Ubuntu Bash | 01-002

```
bash deploy/scripts/probe-gpu.sh
```

1행 CUDA 기본 이미지로 Pod 하나를 띄워 `nvidia-smi -L` 을 찍고, 노드가 광고하는 GPU 수와 Pod 의 사건(Events)을 `results/cloud/01_gpu_probe.txt` 에 남긴다. 스크립트 한 줄씩 읽기는 부록.

관찰 — 둘 중 하나가 나온다

결과	보이는 것	뒤 장의 모양
A. 된다	`GPU 0: NVIDIA GeForce RTX 5070 Laptop GPU (UUID: …)` , 노드 `gpu=1`	07장에서 SGLang 을 **Pod** 로 올린다. 모델 가중치는 PVC(10장)

결과	보이는 것	뒤 장의 모양
B. 안 된다	Pod 가 `Pending` (사건: `Insufficient nvidia.com/gpu`) 또는 `nvidia-smi` 실패, 노드 `gpu=` (빈 값)	07장에서 SGLang 은 **호스트 Docker**(3주차 compose 그대로)에 두고, 클러스터의 게이트웨이가 `host.minikube.internal:30000` 로 붙는다. 실제 클라우드로 치면 'GPU 노드 풀을 따로 둔 것'과 같은 그림

B 가 나와도 실습은 모두 된다 — 모델 서버만 클러스터 밖이다. 어느 쪽이든 결과를 아래 표에 적는다.

장애·복구 · `minikube start` 가 `--gpus` 에서 실패하면 옵션을 빼고 다시 만든다(`minikube delete -p agent` 뒤 `--gpus` 없이). 그 경우는 B 다.

정리 · 이 장은 내리지 않는다 — 02장이 이 클러스터를 쓴다. `kubectl -n lab delete pod gpu-probe` .

성공 확인 · `results/cloud/01_gpu_probe.txt` 가 있고, 아래 표를 채웠다.

항목	결과 (2026-10-12, 노트북)
minikube 버전 / 드라이버	v1.39.0 / docker (Kubernetes v1.37.0, 노드 CPU 4 · 6GB)
`--gpus=all` 로 start 됐나	됐다. 장치 플러그인 `nvcr.io/nvidia/k8s-device-plugin:v0.20.0` 이 `Registered device plugin for 'nvidia.com/gpu'`
노드 `nvidia.com/gpu` 광고 수	**1**
Pod 의 `nvidia-smi -L`	`GPU 0: NVIDIA GeForce RTX 5070 Laptop GPU (UUID: GPU-3e4f…)` — 이미지 받기 6.5초 포함 10초
판정 (A/B)	**A — 모델 서버를 클러스터 안 Pod 로 올린다(07장)**

첫 실행에서는 Pod 가 `Forbidden: serviceaccount "default" not found` 로 거부됐다 — 막 만든 네임스페이스의 기본 서비스 계정이 몇 초 뒤에 생기는데 그 전에 Pod 를 만들었다. 스크립트가 서비스 계정과 장치 플러그인을 기다리게 고쳤다(문제해결 이력 C9). 호스트 드라이버 592.19, Docker 런타임에 `nvidia` 있음.

---

02. 클러스터 둘러보기 — 노드 · 네임스페이스 · 첫 Pod

완료 목표 · 노드 하나짜리 클러스터의 자원과 작업 구역을 읽고, 상담 화면 Pod 하나를 띄워 본다(이미지는 03장에서 만들므로 여기선 공개 이미지로 흐름만).

```
그림 02 | minikube 한 대 = 제어부 + 노드 하나
  kubectl → API 서버 → 스케줄러 → 노드(agent) → Pod(컨테이너)
             └ etcd(상태 저장)                  └ 네임스페이스 lab 안에만 만든다
```

실행

Ubuntu Bash | 02-001

```
kubectl get nodes -o wide
kubectl describe node agent | sed -n '/Allocatable/,/System Info/p'
kubectl get ns
kubectl -n lab run hello --image=python:3.12-slim --restart=Never -- python -c "print('lab ok')"
kubectl -n lab wait --for=jsonpath='{.status.phase}'=Succeeded pod/hello --timeout=120s
kubectl -n lab logs hello
kubectl -n lab delete pod hello
```

1행 노드 목록과 주소·버전. `-o wide`: 내부 IP 와 컨테이너 런타임까지. 2행 노드 상세 중 '할당 가능' 자원(cpu·memory·`nvidia.com/gpu`)만 잘라 본다. 01장의 판정과 같아야 한다. 3행 네임스페이스 목록. `lab` 과 시스템용 `kube-system` 이 보인다. 4행 Pod 하나를 만들어 파이썬 한 줄을 실행한다. `--restart=Never`: 끝나면 그대로 둔다(Job 처럼). `--`: 뒤는 컨테이너 안 명령. 5행 Pod 가 `Succeeded` 될 때까지 기다린다(최대 120초 — 이미지 받기 포함). 6행 컨테이너 출력. `lab ok`. 7행 지운다.

관찰 · `kubectl -n lab get events --sort-by=.lastTimestamp | tail` 로 Pod 하나가 만들어지는 사건 순서(Scheduled → Pulling → Pulled → Created → Started)를 본다. 클라우드의 모든 배포가 이 다섯 단계다.

장애·복구 · 4행에서 `ImagePullBackOff` 면 인터넷(Docker Hub) 문제다. `minikube -p agent ssh -- docker pull python:3.12-slim` 로 노드 안에서 직접 받아 본다.

정리 · 없음(Pod 는 7행에서 지움).

성공 확인 · 2행의 `Allocatable` 에 `memory` 가 6Gi 근처, 6행 출력이 `lab ok`.

---

03. 에이전트 이미지 만들기 — 코드를 상자 하나에

완료 목표 · 7주차의 상담 화면·에이전트 그래프·창구 서버를 담은 컨테이너 이미지 `onprem-agent:v1` 을 만들어 minikube 노드에 넣고, 크기와 만드는 시간을 잰다.



이미지에는 **코드와 의존성만** 넣는다. 모델 서버는 07장의 Pod 가 따로 맡고, 검색 임베더(bge-m3, torch 2GB)도 넣지 않는다 — 넣으면 이미지가 3GB 가 되어 올리고 되돌리는 시간이 다 늘어난다. 03~09장은 해시 임베더로 돌고(규칙 모델과 시험이 쓰는 그대로), 검색 임베더는 10장에서 붙인다. 한 이미지가 **실행 명령만 바꿔** 두 역할을 한다: 상담 화면(`python scripts/web.py`)과 전체 시험(`make test-full`, 12장).

실행

```
Ubuntu Bash | 03-001

1 cat deploy/Dockerfile
2 bash deploy/scripts/03-build.sh v1
3 cat results/cloud/03_image.txt
```

1행 이미지 설계도를 읽는다. 층(layer) 순서가 요점이다 — `pyproject.toml` 과 `src` 를 먼저 복사해 `pip install` 하고, 자주 바뀌는 `scripts` · `examples` · `data` 는 뒤에 복사한다. Docker 는 바뀌지 않은 층을 캐시에서 재사용하므로 코드만 고치면 두 번째 빌드는 설치를 건너뛴다. 마지막에 일반 사용자(`agent`)로 바꾸고 `HEALTHCHECK` 로 `/healthz` 를 찍는다. 2행 만들고(`docker build`), 호스트 Docker 에서 한 번 띄워 `/healthz` 가 200 인지 보고, `minikube image load` 로 노드 안 창고에 넣는다. 시간과 크기를 `results/cloud/03_image.txt` 에 남긴다. 한 줄씩 읽기는 부록 A3. 3행 결과를 읽는다. `build:` 초, `size:` 바이트, 층 수, `/healthz` 응답(`{"ok":true,"pod":"…"}`), `load into minikube:` 초.

관찰 · `docker history onprem-agent:v1` 에서 가장 큰 층은 `pip install` 층이다(langgraph · langchain · fastapi · 의존성). 코드 층은 몇 MB 다. 두 번째 빌드(`bash deploy/scripts/03-build.sh v1` 을 바로 다시)는 설치 층을 캐시에서 가져와 수 초에 끝난다 — 그 시간도 같은 파일에 쌓이니 둘을 견준다.

장애·복구 · `pip install` 에서 멈추면 WSL2 의 DNS 문제가 잦다 — `docker build --network=host` 로 다시. `minikube image load` 가 수 분 넘게 걸리면 `minikube -p agent image build -t onprem-agent:v1 -f deploy/Dockerfile .` 로 노드 안에서 직접 만든다(호스트 캐시는 못 쓴다).

정리 · 없음(이미지는 04장이 쓴다). 호스트 쪽 확인용 컨테이너 `agent-check` 는 스크립트가 지운다.

성공 확인 · `minikube -p agent image ls | grep onprem-agent` 에 `docker.io/library/onprem-agent:v1`, 아래 표를 채웠다.

항목	결과 (2026-10-12, 노트북, 커밋 446d794)
이미지 크기 / 층 수	**208MB** / 17층 — `pip install` 층 383MB(압축 전), 코드 층은 다 합쳐 2MB 안팎
처음 빌드 시간 / 코드 그대로 두 번째 빌드 시간	**43초 / 1초** (캐시)

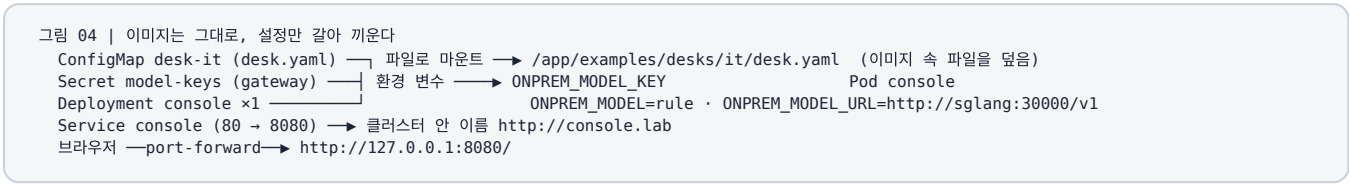
항목	결과 (2026-10-12, 노트북, 커밋 446d794)
`/healthz` (호스트 Docker)	`{"ok":true,"pod":"da212bfccd4a","model":"rule"}`
minikube 에 넣는 시간	11초, 두 번째 6초

(두 번째는 코드를 바꾸지 않고 돌린 것이라 '코드만 바꾼' 경우보다 빠르다 — 코드를 고치면 `COPY` 층부터 다시 만들지만 설치 층은 여전히 캐시다.)

---

04. 설정과 비밀을 밖으로 — ConfigMap · Secret · 상담 화면 Pod 하나

완료 목표 · 창구 정의(`desk.yaml`)를 ConfigMap 으로, 모델 키를 Secret 으로 밖에 두고, 상담 화면 Pod 하나를 띄워 브라우저로 연다. 이미지를 다시 만들지 않고 화면 문구가 바뀌는 것을 본다.



7주차 프레임워크의 핵심이 여기서 그대로 쓰인다 — **창구 하나 = 폴더 하나(`desk.yaml`)** 라서, 창구 설정을 이미지 밖 ConfigMap 으로 뺄 수 있다. 실제 클라우드에서 '설정은 이미지에 굽지 않는다'는 원칙과 같다. 모델은 이 장과 05장에서 **규칙 모델**(GPU 없이, 시험 때 쓰는 것)이다 — 화면과 배포 흐름만 보고, 실제 모델은 07장에서 붙인다.

실행

Ubuntu Bash | 04-001

```
bash deploy/scripts/04-config.sh
kubectl apply -f deploy/manifests/04-console.yaml
time kubectl -n lab rollout status deploy/console --timeout=180s
kubectl -n lab get pods,svc -o wide
kubectl -n lab port-forward svc/console 8080:80
```

1행 ConfigMap `desk-it` 를 `examples/desks/it/desk.yaml` 파일에서, Secret `model-keys` 를 환경 변수 `GATEWAY_KEY` (없으면 5주차 개발용 키)에서 만든다. `--dry-run=client -o yaml | kubectl apply` 꼴이라 다시 돌려도 덮어쓴다. 부록 A4. 2행 Deployment(상담 화면 Pod 의 설계도, 지금은 1개)와 Service(클러스터 안 이름)를 만든다. 매니페스트 해설은 아래. 3행 Pod 가 준비(Ready)될 때까지의 시간을 잰다 — `readinessProbe` (`/readyz`)가 200 을 줘야 Ready 다. 4행 Pod 의 IP·노드와 Service 의 ClusterIP 를 본다. 5행 내 컴퓨터의 8080 을 Service 의 80 에 잇는다. 브라우저에서 `http://127.0.0.1:8080/` (상담 화면) 과 `/admin` (관리자 화면). 끝낼 땐 Ctrl+C.

`deploy/manifests/04-console.yaml` 의 요점:

항목	뜻
`imagePullPolicy: Never`	03장에서 노드에 직접 넣은 이미지라 밖(Docker Hub)에서 받지 않는다. 빠뜨리면 `ErrImageNeverPull` / `ImagePullBackOff`
`env ONPREM_MODEL=rule`	어느 모델로 돌지. 07장에서 `sglang-4b` 로 바꾼다. `ONPREM_MODEL_URL` 은 07장 모델 서버의 Service 이름
`secretKeyRef model-keys/gateway`	키는 매니페스트에 적지 않고 Secret 에서 환경 변수로
`volumeMounts … subPath: desk.yaml`	ConfigMap 의 파일 하나만 이미지 속 같은 자리에 덮는다(폴더째 덮으면 `prompt.md` · `server.py` 가 사라진다)
`emptyDir /app/var`	대화·승인 대기(SQLite). **Pod 가 사라지면 같이 사라진다** — 05장이 이걸 잰다
`livenessProbe /healthz` · `readinessProbe /readyz`	살아 있나(아니면 다시 띄움) · 받을 준비가 됐나(아니면 Service 가 요청을 안 보냄). `/readyz` 는 창구 정의를 읽을 수 있고 실제 모델이면 엔진에 닿아야 200

항목	뜻
`resources requests/limits`	노드 6GB 중 이 Pod 몫(메모리 768Mi~1Gi — 처음 384Mi 로 적었다가 06장 측정 뒤 올림). 06장의 HPA 가 `requests.cpu` 를 기준 삼는다

관찰 — 설정만 바꾸기

Ubuntu Bash | 04-002

```
  1 sed 's/placeholder: 사번, 장비 번호, 오류 코드… 무엇이든 물어보세요/placeholder: (ConfigMap 에서 바꾼 문구) 무엇이든 물어보세요/' examples/desks/it/desk.yaml > /tmp/desk.yaml
  2 bash deploy/scripts/04-config.sh /tmp/desk.yaml
  3 kubectl -n lab rollout restart deploy/console && kubectl -n lab rollout status deploy/console
  4 curl -s http://127.0.0.1:8080/api/desks | python3 -m json.tool | grep placeholder
```

1행 창구 정의의 입력칸 문구만 바꾼 사본을 만든다(저장소 파일은 그대로). 2행 ConfigMap 을 그 사본으로 덮는다. 3행 Pod 를 새로 띄운다 — ConfigMap 을 `subPath` 로 마운트하면 바뀐 내용이 자동으로 들어오지 않으므로 재시작이 필요하다(실제 서비스에서는 체크섬 주석으로 자동 재시작한다, 13장). 4행 (5행의 port-forward 를 다시 켠 뒤) 상담 화면이 받는 창구 목록에 바뀐 문구가 보인다. 이미지는 그대로, 설정만 갈렸다. 끝나면 `bash deploy/scripts/04-config.sh` 로 되돌린다.

장애·복구 · `port-forward` 가 `address already in use` 면 내 컴퓨터의 8080 을 누가 쓰고 있다(7주차 `scripts/web.py` 가 떠 있거나 지난 port-forward 가 남음) — `kubectl -n lab port-forward svc/console 18080:80` 으로 다른 포트에 잇고 브라우저는 `http://127.0.0.1:18080/` . · Pod 가 `CrashLoopBackOff` 면 `kubectl -n lab logs deploy/console --previous` 로 죽기 직전 출력을 본다. 흔한 원인: ConfigMap 의 `desk.yaml` 이 잘못돼 창구를 못 읽음(`/readyz` 503) · Secret 이 없음(`CreateContainerConfigError`). `kubectl -n lab describe pod -l app=console` 의 Events 가 어느 쪽인지 말해 준다.

정리 · Ctrl+C 로 port-forward 를 끊는다. Deployment 는 05장이 쓰니 둔다.

성공 확인 · 3행이 `successfully rolled out` , 브라우저에서 IT 헬프데스크 창구로 "VPN-809 오류가 떠요" 를 물으면 규칙 모델이 규정을 찾아 답한다(관리자 화면에 모델 `rule`). 표를 채운다.

항목	결과 (2026-10-12, 노트북)
apply → Ready 시간 (3행 `real`)	**5.9초** (이미지가 이미 노드에 있어 받기 0초). 첫 시도의 port-forward 는 8080 충돌로 실패 → 18080 으로 열어 브라우저 확인
Pod 의 메모리 (`kubectl -n lab top pod` — metrics-server 애드온이 있을 때)	06장에서 metrics-server 를 켠 뒤 잰다
ConfigMap 문구 바꾸기 → 화면 반영 (04-002)	**됨** — 재시작 뒤 `/api/desks` 의 IT 창구 입력칸이 `(ConfigMap 에서 바꾼 문구) 무엇이든 물어보세요` 로. 이미지는 그대로. 재시작(rollout restart → 완료) 수 초

05. 나눠 주기와 무상태 — 상담 화면 ×3, 로드 밸런서, Pod 를 죽여 보기

완료 목표 · 상담 화면 Pod 를 3개로 늘려 Service 가 요청을 나눠 주는 것을 세고, 승인을 기다리는 상담 중에 그 Pod 를 죽여 **상태가 Pod 안에 있으면 무슨 일이 생기는지** 본다.

```
그림 05 | Service 는 라벨이 같은 Pod 전부에 돌아가며 보낸다
                    ┌─▶ console-xxx-a  (var/web_state.db  ← 상담 A 의 승인 대기)
  http://console.lab ─┼─▶ console-xxx-b  (var/web_state.db  ← 비어 있음)
                    └─▶ console-xxx-c  (var/web_state.db  ← 비어 있음)
  같은 상담의 두 번째 요청(승인)이 b 에 가면? ─▶ "확인을 기다리는 일이 없다"(409)
```

잰다 — 결과를 보기 전에 예상을 적는다. 로드 밸런서: 30번 부르면 세 Pod 가 10번 안팎씩 받을 것이다(완전히 같지는 않다 — Service 는 무작위에 가깝다). 무상태: 04장의 설계대로면 대화·승인 대기가 **Pod 마다 따로**(emptyDir 의 SQLite) 있으므로, 질문을 받은 Pod 가 죽거나 승인 요청이 다른 Pod 에 가면 이어지지 않을 것이다 — 즉 **지금 상담 화면은 무상태가 아니다.** 이걸 눈으로 확인하는 게 이 장이고, 고치는 것(대화 저장을 공용 DB 로)이 10장이다.

실행

Ubuntu Bash | 05-001

```
kubectl -n lab scale deploy/console --replicas=3 && kubectl -n lab rollout status deploy/console
kubectl -n lab get endpoints console
bash deploy/scripts/05-lb.sh 30
bash deploy/scripts/05-kill.sh
kubectl -n lab get pods -l app=console
```

1행 복제 수를 3으로. 같은 이미지·같은 설정의 Pod 가 둘 더 뜬다. 2행 Service 뒤에 매달린 Pod 주소 세 개. 이 목록이 로드 밸런서의 '대상 그룹'이다. 3행 클러스터 안에 curl Pod 를 띄워 `http://console.lab/healthz` 를 30번 부르고, 답 머리말 `X-Served-By`(답한 Pod 이름)를 센다. `kubectl port-forward` 는 Pod 하나에 고정되므로 쓰지 않는다. 결과 `results/cloud/05_lb.txt`. 부록 A5. 4행 ① 상담 번호 하나로 "E1063 계정 잠금 해제 요청해 줘…" 를 보내 담당자 확인(approval 사건)에서 멈추게 하고 어느 Pod 가 받았는지 적는다 → ② 그 Pod 를 지운다 → ③ 같은 상담 번호로 승인을 보낸다. 결과 `results/cloud/05_kill.txt`. 부록 A5. 5행 지운 Pod 자리에 새 Pod 가 떠 있다(Deployment 가 3개를 지킨다).

관찰 · 4행에서 승인이 `HTTP 409 확인을 기다리는 일이 없다` 로 끝나면 예상대로다 — 승인 대기가 죽은 Pod 의 SQLite 에만 있었다. Pod 를 지우지 않아도 승인 요청이 다른 Pod 에 가면(세 개 중 둘) 같은 409 다. 실제 클라우드에서 '앱 서버는 무상태여야 한다'는 말의 뜻이 이것이다: 서버 안에 남는 것이 있으면 늘리지도, 바꾸지도, 죽이지도 못한다.

장애·복구 · 당장의 임시 처방은 **같은 사람을 같은 Pod 로** 보내는 것이다: `kubectl -n lab patch svc console -p '{"spec":{"sessionAffinity":"ClientIP"}}'`. 브라우저 한 대로 쓰는 동안은 이어지지만, 그 Pod 가 죽으면 역시 잃는다(05-kill 로 확인). 제대로 된 처방은 10장 — 대화·승인 대기를 PostgreSQL 체크포인터로 옮겨 어느 Pod 가 받아도 같은 상담을 잇는다. 확인 뒤 되돌린다: `kubectl -n lab patch svc console -p '{"spec":{"sessionAffinity":"None"}}'`.

정리 · `kubectl -n lab scale deploy/console --replicas=1` (06장이 1 에서 시작한다).

성공 확인 · `results/cloud/05_lb.txt` 에 Pod 이름 셋과 횟수, `05_kill.txt` 에 지운 Pod 이름과 승인 응답. 표를 채운다.

항목	예상	결과 (2026-10-12, 노트북)
30회 요청의 Pod 별 횟수	10 안팎 × 3	**9 · 8 · 10** (`f77qv` · `nhzqj` · `nzsdm`) — 셋이 27회고 **3회는 Pod 이름 없이** 돌아왔다. 측정기가 HTTP 코드를 안 찍어 원인을 못 가렸다(1회용 Pod 의 첫 요청이 DNS 전에 나갔을 가능성). 스크립트가 코드를 같이 찍게 고쳐 06장에서 다시 세니 `26 200 …` — 이번엔 4회가 없고 오류도 없었다. 원인은 `kubectl run -i` 가 붙기 전의 앞 줄을 잃는 것(D45) — 요청은 30번 다 갔다. logs 로 읽게 고치니 **30/30**(06-002)
승인 대기 중 Pod 삭제 → 같은 상담 번호 승인	409 (이어지지 않음)	**409 `확인을 기다리는 일이 없다`** — 질문은 `f77qv` 가 받아 담당자 확인에서 멈췄고, 그 Pod 를 지운 뒤 승인은 새 Pod `4xtcm` 이 받아 모른다고 답함
새 Pod 가 다시 3개가 되기까지	수 초	승인 요청 시점(삭제 3초 뒤)에 이미 새 Pod 가 답했다
결론	지금 상담 화면은 무상태가 아니다 → 10장	**예상대로.** 승인 대기는 죽은 Pod 의 SQLite 에만 있었다

측정 원본: reports/cloud_w8.md.

06. 자동으로 늘리고 줄이기 — metrics-server · HPA · 부하

완료 목표 · 상담 화면 Pod 를 CPU 사용률로 1 → 4 개까지 자동으로 늘리는 HPA 를 걸고, 부하를 주어 늘어나는 과정과 부하가 끝난 뒤 줄어드는 과정을 10초 간격으로 적는다.

```
그림 06 | HPA 는 '평균 CPU 가 목표를 넘으면 Pod 를 더' — 측정은 metrics-server 가 한다
  부하 Pod ──▶ Service console ──▶ console ×1 … ×4
                                    ▲ 10초마다 CPU 를 읽음(metrics-server)
  HPA: 목표 평균 50% of requests.cpu(200m) · 넘으면 15초마다 +2 · 밑돌면 60초 안정 뒤 줄임
  노드 자동 확장(노드 자체를 더)은 없다 — 노트북 한 대. 실제 클라우드에선 Cluster Autoscaler/Karpenter 가 같은 원리로 노드를
```

HPA 는 `requests.cpu` 를 100% 로 본다. 04장의 상담 화면 Pod 는 `requests.cpu: 200m` 이므로 목표 50% 는 Pod 당 100m — 규칙 모델 질문 하나가 그래프 + 창구 서버(MCP)를 거쳐 수십 ms 의 CPU 를 쓰니, 동시 8개를 계속 보내면 넘는다. **예상:** 30~60초 안에 Pod 가 1 → 3~4 로 늘고, 부하가 끝나고 60초 안정 뒤 1 로 돌아온다. 처음 10~20초는 HPA 가 `<unknown>` 을 보인다(metrics-server 가 첫 수치를 모으는 시간).

실행

Ubuntu Bash | 06-001

```
1 minikube -p agent addons enable metrics-server
2 kubectl -n kube-system rollout status deploy/metrics-server --timeout=120s
3 kubectl -n lab top pod
4 kubectl apply -f deploy/manifests/06-hpa.yaml
5 kubectl -n lab get hpa console
6 bash deploy/scripts/05-lb.sh 30
7 bash deploy/scripts/06-load.sh 150 8
```

1행 metrics-server 를 켠다 — 노드·Pod 의 CPU·메모리를 15초마다 모아 `kubectl top` 과 HPA 에 준다. 실제 클라우드의 CloudWatch 지표 수집에 해당. 2행 뜰 때까지 기다린다. 3행 상담 화면 Pod 의 지금 CPU·메모리(04장 표의 빈칸이 여기서 채워진다). 처음 1분은 `metrics not available yet` 일 수 있다. 4행 HPA 를 만든다. 매니페스트 해설은 위 그림. 5행 `TARGETS` 가 `<unknown>/50%` 에서 곧 `n%/50%` 로 바뀐다. 6행 05장에서 30회 중 3회가 Pod 이름 없이 돌아온 것을, HTTP 코드를 찍게 고친 측정기로 다시 센다(replicas 는 1 이라 전부 한 Pod 가 받는다 — 빠진 요청이 있는지만 본다). 7행 부하 150초(동시 8)를 걸고 10초마다 HPA 의 현재 %, 목표 Pod 수, 준비된 Pod 수를 적는다. 부하가 끝난 뒤 90초를 더 보아 줄어드는 것까지 적는다. 결과 `results/cloud/06_hpa.txt` . 부록 A6.

관찰 · 표에서 세 시점을 찾는다 — 현재 % 가 50 을 처음 넘은 때, Pod 수가 처음 늘어난 때(그 차이가 HPA 의 반응 시간), 부하 종료 뒤 Pod 가 1 로 돌아온 때(60초 안정 창 + 측정 주기). `kubectl describe hpa console` 의 Events 에 `SuccessfulRescale … above target` 과 `… below target` 이 보인다.

장애·복구 · `TARGETS` 가 계속 `<unknown>` 이면 metrics-server 가 Pod 를 못 읽는 것이다 — `kubectl -n kube-system logs deploy/metrics-server | tail` 에 `x509` 가 보이면 minikube 애드온은 이미 `--kubelet-insecure-tls` 를 넣어 주지만, 아니면 `kubectl -n kube-system edit deploy metrics-server` 로 그 인자를 더한다. Pod 가 4 까지 늘지 않고 2~3 에서 멈추면 부하보다 Pod 가 빨리 처리하는 것이다(좋은 일) — 7행의 동시 수를 16 으로 올려 본다.

정리 · `kubectl delete -f deploy/manifests/06-hpa.yaml` (HPA 를 두면 07장에서 모델 서버가 노드 메모리를 쓰는 동안 상담 화면이 늘어나 겹칠 수 있다). Pod 는 1 로 둔다.

성공 확인 · `results/cloud/06_hpa.txt` 에 Pod 수가 1 보다 커진 줄과 다시 1 이 된 줄이 모두 있다.

항목	예상	결과 (날짜, 노트북)
상담 화면 Pod 의 평시 CPU · 메모리 (`top pod`)	수 m · 200~300Mi	**4m · 670~764Mi** — 메모리가 예상의 두 배 넘게 크다(langchain·langgraph·창구 서버 적재). `requests.memory: 384Mi` 는 작게 잡은 것 — 4개면 2.9GB, 07장 전에 `requests` 를 768Mi 로 올린다
05장 다시 세기: 30회 중 HTTP 200 과 Pod 이름이 찍힌 수	30/30	26/30(두 번째 측정기) → **30/30**(logs 로 읽는 세 번째 측정기, 06-002) — 빠진 건 측정기였다(D45)
50% 를 처음 넘은 뒤 Pod 가 늘기까지	15~30초	부하 시작 09:23:19 → 지표는 60초 동안 2% 에 머물다 09:24:21 에 **315%** 로 한 번에 뛰며 그 순간 Pod 3 → 10초 뒤 4. 즉 **지표가 약 60초 늦게** 오고, 오자마자 HPA 는 바로 늘렸다(한 단계 +2, 다음 +1 = 상한 4)
부하 중 최대 Pod 수	3~4	**4**(상한). 150초에 9,376 요청(초당 62), Pod 당 CPU 630m(315%)
부하 끝 → 1 로 돌아오기까지	60~90초	**못 봄** — 부하가 09:25:49 에 끝났는데 관찰이 09:27:16 까지였고 지표는 아직 115%(늦게 오는 지표가 내려오는 중). 줄어드는 데는 '지표 지연 60초 + 안정 창 60초' 이상이 든다. 스크립트가 부하 뒤 240초를 보게 고쳐 07장 전에 한 번 더(아래 06-002)

Ubuntu Bash | 06-002 (줄어드는 것까지 보기 — 07장 전에)

```
1 kubectl apply -f deploy/manifests/06-hpa.yaml
2 bash deploy/scripts/06-load.sh 90 8 240
3 kubectl delete -f deploy/manifests/06-hpa.yaml
```

1~3행 같은 HPA 로 부하 90초 뒤 240초를 더 본다. 표에 '부하 끝 → 1' 시간을 채운다.

06-002 항목	결과 (2026-10-12, 노트북)
부하 끝 → Pod 1 로	**106초** — 부하 09:48:51~09:50:21, 지표 38% 가 09:51:26 에 처음 보이고(지연 약 60초) 60초 안정 창 뒤 09:52:07 에 4 → 1. Events `New size: 1; reason: All metrics below target`
늘어나기(다시)	09:50:24 에 3, 09:50:45 에 4 — 첫 측정과 같은 모양. 부하 90초에 4,152 요청. 처음 60초의 빈 지표는 직전 `apply`(메모리 768Mi)로 Pod 가 새로 떠서 `pods might be unready`
05장 다시 세기 (logs 로 읽는 측정기)	**30/30, 전부 200** — D45 확인. 빠졌던 3~4회는 요청이 아니라 측정기였다

07. 모델 서버를 클러스터 안으로 — GPU Pod · PVC · 상담 화면을 실제 모델로

완료 목표 · 01장 판정 A 에 따라 SGLang(Qwen3-4B(FP8))을 GPU 를 요구하는 Pod 로 올리고, 가중치는 PVC 에 두며, 상담 화면이 규칙 모델 대신 이 모델 서버를 쓰게 바꾼다. 그리고 **Pod 로 올려서 느려졌는지** 3주차 측정기로 호스트 Docker 와 견준다.



옮길 큰 파일이 둘이다. 모델 서버를 띄우려면 **프로그램**과 **재료**가 모두 노드 안에 있어야 한다. 프로그램은 SGLang 이미지(CUDA·PyTorch 포함, 10GB 급) — 3주차에 노트북 Docker 에 받아 두었지만 minikube 노드는 자기 창고가 따로라 `minikube image load` 로 복사한다(2행). 재료는 Qwen3-4B(FP8) 가중치(모델이 배운 내용이 든 숫자 파일, 4.4GB) — 1회용 Job 이 Hugging Face 에서 PVC(클러스터 안 저장 공간)로 한 번 받아 두면, 모델 서버 Pod 가 죽었다 떠도 다시 받지 않는다(4행). 03장에서 에이전트 이미지에 모델을 넣지 않은 것과 같은 이유로, 프로그램과 재료를 떼어 둔다. 비유: 식당을 열려면 주방 설비(이미지)와 재료(가중치)가 다 건물(노드) 안에 있어야 하고, 설비를 바꿀 때 재료까지 다시 사 오지 않는다.

메모리 먼저. 01장에서 노드를 `--memory=6g` 로 만들었다 — docker 드라이버에서 이 숫자는 노드 컨테이너의 Docker 메모리 상한이다. SGLang 은 GPU 외에 호스트 RAM 을 3~4GB 쓰므로 상담 화면과 합치면 6GB 를 넘길 수 있다. 클러스터를 다시 만들지 않고 **`docker update` 로 상한만 올린다**(1행) — WSL2 의 15GB 중 10GB 를 노드에, 나머지는 벤치마크·브라우저 몫. 02장 2행에서 본 노드의 `Allocatable memory` 가 호스트 전체(15Gi 근처)로 보였다면 kubelet 은 이 상한을 모른다는 뜻이다 — 스케줄러는 막지 않고, 넘으면 노드 컨테이너 안에서 OOM 으로 죽는다. 그래서 이 장은 **상담 화면 1개 + 모델 서버 1개**만 띄운다.

실행

Ubuntu Bash | 07-001

```
  1 docker update --memory=10g --memory-swap=10g agent && docker inspect agent --format
'{{.HostConfig.Memory}}'
  2 docker images lmsysorg/sglang:latest --format '{{.Size}}'
  3 kubectl apply -f deploy/manifests/07-sglang.yaml
  4 kubectl apply -f deploy/manifests/07-download.yaml && time kubectl -n lab wait --
for=condition=complete job/download-qwen3-4b --timeout=1800s
  5 kubectl -n lab logs job/download-qwen3-4b | tail -3
  6 time kubectl -n lab rollout status deploy/sglang --timeout=1800s
  7 kubectl -n lab logs deploy/sglang | grep -E "cache|graph|Uvicorn|ready" | tail -5
  8 kubectl -n lab set env deploy/console ONPREM_MODEL=sglang-4b && kubectl -n lab
rollout status deploy/console
  9 kubectl -n lab port-forward svc/console 18080:80
```

1행 노드 컨테이너(이름 `agent`)의 메모리 상한을 10GB 로. 출력 `10737418240`. 클러스터·Pod 는 그대로 산다. `minikube stop/start` 를 하면 노드 컨테이너가 다시 만들어져 6g 로 돌아가므로 start 뒤 다시 건다. 2행 3주차에 받아 둔 SGLang 이미지의 크기만 본다(10GB 급). 처음엔 03장처럼 `minikube image load` 로 노드에 복사하려 했는데 **WSL2 가 통째로 재시작됐다**(C10) — 10GB 를 임시 파일로 풀며 메모리를 넘긴 것으로 보인다. 그래서 이 이미

지는 **노드가 Docker Hub 에서 직접 받는다**(`imagePullPolicy: IfNotPresent`, 3행의 apply 가 받기 시작). 인터넷 속도에 달렸고 6행의 기다림에 포함된다. 3행 PVC(20Gi) + 모델 서버 Deployment + Service 를 만든다. 가중치가 아직 없으니 Pod 는 startupProbe 를 기다리며 실패를 거듭한다 — 괜찮다, 4행이 끝나면 다음 재시작에 뜬다. (`HF_HUB_OFFLINE=1` 이라 인터넷에서 받으려 하지 않는다.) 4행 가중치 내려받기 Job. `python:3.12-slim` 에 `huggingface_hub` 를 깔고 `hf download Qwen/Qwen3-4B-FP8` 를 PVC 로. 4.4GB 라 인터넷 속도에 달렸다 (100Mbps 면 6분). 시간을 적는다. 호스트 `~/.cache/huggingface` 에 이미 있는 파일을 재활용하는 길은 장애·복구. 5행 `du -sh` 로 받은 크기. 6행 모델 서버가 Ready 될 때까지 — 이미지 받기(10GB) + 가중치 읽기 + CUDA 그래프 준비. 시간을 적는다. 다른 창에서 `kubectl -n lab describe pod -l app=sglang | tail -5` 로 `Pulling` → `Pulled` 를 본다. 7행 접두사 캐시(`enable-cache-report`)·CUDA 그래프 줄이 보이면 3주차와 같은 설정이다. 8행 상담 화면의 모델을 `sglang-4b` 로 바꾼다 — `ONPREM_MODEL_URL=http://sglang:30000/v1` (04장 매니페스트)로 붙는다. `/readyz` 가 모델 서버에 닿아야 Ready 가 되므로 6행이 먼저다. 9행 브라우저 `http://127.0.0.1:18080/` — 이제 **실제 모델**이 답한다. IT 헬프데스크에 "E1063 계정 잠금 해제 요청해 줘. 비밀번호를 다섯 번 틀렸대요. 본인 확인은 끝났어요." 를 넣으면 담당자 확인 카드가 뜨고, 승인하면 요청이 실행된다. 관리자 화면에 모델 `sglang-4b`, 엔진 `닿음`.

잰다 — Pod 로 올려서 느려졌나 (서빙 측정기 그대로, 생각 모드 끔, 동시 1·4·8)

Ubuntu Bash | 07-002

```
kubectl -n lab port-forward svc/sglang 30000:30000 &
python scripts/bench_serving.py configs/serving_cloud_pod.yaml
kill %1; kubectl -n lab scale deploy/sglang --replicas=0 && kubectl -n lab scale deploy/console --replicas=0
python scripts/bench_serving.py configs/serving_cloud_host.yaml --manage
kubectl -n lab scale deploy/sglang --replicas=1 && kubectl -n lab scale deploy/console --replicas=1
cat results/serving_cloud_pod.md results/serving_cloud_host.md
```

1~2행 Pod 를 호스트의 30000 으로 잇고 잰다(결과 `results/serving_cloud_pod.md`). port-forward 가 한 겹 더 끼므로 첫 글자 시간에 몇 ms 가 얹힌다 — 이건 측정 경로의 비용이고 Pod 의 비용이 아니다. 둘을 가르려면 3행 뒤 `kubectl -n lab run` 으로 클러스터 안에서 재야 하는데, 이 장에서는 port-forward 로 재고 그 사실을 적는다. 3행 GPU 는 하나다 — Pod 를 내려 GPU 를 비운다(상담 화면도 `/readyz` 가 실패하므로 같이). 4행 바탕: 3주차 compose 의 `sglang-4b` 프로필을 `--manage` 로 띄워 같은 요청으로 잰다(결과 `results/serving_cloud_host.md`). 끝나면 측정기가 내린다. 5행 되돌린다. 6행 두 표를 나란히 읽는다.

예상: 첫 글자 시간(TTFT)·토큰당 시간(TPOT)·처리량 모두 **5% 안**에서 같다. (단, Pod 쪽은 C11 로 attention·FP8 행렬곱 백엔드가 triton 이고 호스트는 3주차의 옛 이미지·기본 백엔드라, 차이가 나면 'Pod 의 비용'이 아니라 '백엔드 차이'일 수 있다 — 호스트도 같은 인자로 돌리는 게 맞는 비교이며 `serving_cloud_host.yaml` 의 `SGLANG_EXTRA` 로 맞춘다.) GPU 계산은 같은 드라이버·같은 커널이고, 컨테이너 한 겹(Pod)은 CPU 쪽 오버헤드만 더한다. 접두사 캐시 적중률도 같아야 한다(같은 요청 묶음).

장애·복구 · (가) Pod 가 `Pending` 에 `Insufficient nvidia.com/gpu` 면 01장의 장치 플러그인이 내려간 것 — `kubectl -n kube-system rollout restart ds/nvidia-device-plugin-daemonset`. (나) `OOMKilled` 또는 노드가 `NotReady` 면 메모리 — 1행의 상한을 12g 로, 또는 `--mem-fraction-static 0.80` 은 그대로 두고 상담 화면을 0 으로. (다) 가중치 내려받기가 느리면 호스트 캐시를 쓴다: `minikube -p agent mount ~/.cache/huggingface:/hf &` 를 켜 두고 Job 대신 `kubectl -n lab run cp --image=busybox --restart=Never --overrides='…hostPath /hf…' -- cp -r /hf/hub/models--Qwen--Qwen3-4B-FP8 /models/hf/hub/` — 9p 라 느리지만 인터넷보다 빠를 수 있다. (라) 6행이 900초를 넘기면 `kubectl -n lab`

`logs deploy/sglang` 의 마지막 줄 — `CUDA out of memory` 면 노트북에서 다른 GPU 프로그램(3주차 compose)이 떠 있는 것이다. `docker compose -f serving/compose.yml --profile sglang-4b down`.

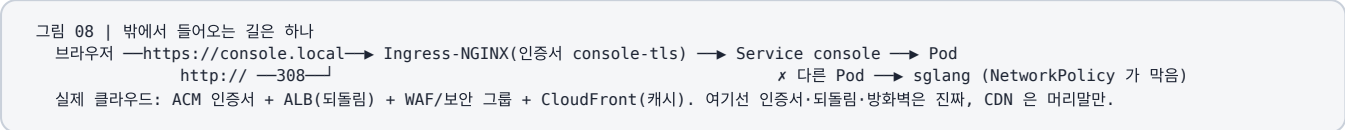
정리 · 모델 서버는 08장 이후에도 쓰니 둔다. 다른 장을 돌리는 동안 GPU 를 비우려면 `kubectl -n lab scale deploy/sglang --replicas=0`.

성공 확인 · 브라우저에서 실제 모델이 답하고 담당자 확인이 뜬다. 아래 표를 채웠다.

항목	예상	결과 (2026-10-12, 노트북)
SGLang 이미지 크기 / 노드가 받는 시간	10GB 급 / 인터넷 속도	노드 안에서 **36.4GB**(압축 풀린 크기) / 받기는 WSL 재시작 전후에 걸쳐 끝나 못 잼
가중치 내려받기(Job) 시간 / 크기	인터넷 속도 / 4.4GB	**2분 47초**(Job 완료까지, 두 번 실패 뒤 세 번째 Pod 가 성공 — 앞 둘은 WSL 재시작 때)
모델 서버 Ready 까지 (6행)	1~3분	**첫 시작: 약 2시간 40분의 사투**(C11) — FP8 커널 즉석 컴파일이 메모리를 넘겨 OOM 반복, 점검 시간 초과. 설정 네 가지를 바꿔 뜸. **두 번째 시작(캐시 있음): 58초**
노드 메모리: 모델 서버 혼자 안정 / 상담 화면과 함께	4~5GB(00장 예산)	**7.9GB / 11.1GB** — 예산이 틀렸다. 00장 예산표를 고침
상담 화면 → 모델 서버 전환 (8행) 뒤 실제 답	담당자 확인 카드	**됨** — "E1063 계정 잠금 해제…" 에 잠금 해제 요청 승인 카드(사번 E1063, 사유 '비밀번호 5회 오류로 잠김, 본인 확인 완료'). 관리자 화면: 모델 qwen3-4b, 주소 http://sglang:30000/v1, 생각 모드 켬
TTFT · TPOT · 처리량: Pod 대 호스트 Docker (동시 1·4·8)	5% 안	
접두사 캐시 적중률: Pod 대 호스트	같음	

08. 엣지 — https · 되돌림 · 방화벽 · CDN 흉내

완료 목표 · 상담 화면 앞에 Ingress-NGINX 를 두고 자체 서명 인증서로 https 를 켠다. http 는 https 로 되돌리고, 모델 서버는 상담 화면(과 09장의 게이트웨이)만 부를 수 있게 NetworkPolicy 를 건다. 정적 파일에 캐시 머리말을 붙여 두 번째 요청이 304 인지 본다.



실행

Ubuntu Bash | 08-001

```
minikube -p agent addons enable ingress
bash deploy/scripts/08-edge.sh
grep -q console.local /etc/hosts || echo "$(minikube -p agent ip) console.local" | sudo tee -a /etc/hosts
```

1행 Ingress-NGINX 컨트롤러(ingress-nginx 네임스페이스)를 켠다 — 클러스터 밖에서 들어오는 http(s) 를 받아 Service 로 넘기는 문지기. 2행 인증서 만들기(`openssl` , `var/console.crt·key`) → Secret `console-tls` → `08-ingress.yaml` (Ingress + NetworkPolicy) → 측정 다섯 가지를 `results/cloud/08_edge.txt` 에. 부록 A8. 3행 브라우저로 보려면 `console.local` 이 minikube IP 를 가리켜야 한다. 그 뒤 `https://console.local/` (자체 서명이라 경고를 한 번 넘긴다).

관찰 · ① http 는 `308` 과 `https://…` 로 되돌림 ② https 는 200, 인증서 `CN=console.local` ③ `/static/app.css` 두 번째 요청이 `304` (ETag) 이고 `Cache-Control: public, max-age=3600` ④ 라벨 없는 Pod 에서 모델 서버는 **막히나**(`000` , exit 28) — **minikube 기본 CNI 는 NetworkPolicy 를 강제하지 않는다.** 막히지 않으면(200) 그게 측정 결과다: 실제 클라우드의 CNI(Calico·Cilium, EKS 의 VPC CNI 정책)는 강제한다. 강제해 보려면 장애·복구. ⑤ 상담 화면 Pod 에서는 200.

장애·복구 · NetworkPolicy 가 안 막히면 `minikube start -p agent --cni=calico` 로 **클러스터를 다시 만들어야** 한다(CNI 는 만들 때 정해진다) — 10GB 이미지와 PVC 를 잃으므로 이 워크북에선 하지 않고 적어 둔다. Ingress 가 `503` 이면 Service 이름·포트가 틀린 것, `404` 면 host 가 안 맞는 것(`--resolve` 또는 `/etc/hosts`).

정리 · 없음(09장이 이 Ingress 를 바꿔 쓴다).

성공 확인 · 아래 표.

항목	예상	결과 (2026-10-12, 노트북)
http → https 되돌림	308	**308** → `https://console.local/healthz`
https 상태 · 인증서	200 · CN=console.local, 365일	**200** · `CN=console.local` , 2026-10-12 ~ 2027-10-12
정적 파일 두 번째 요청	304 + Cache-Control	**304**(ETag). `Cache-Control` 은 안 보임 — 그 코드는 이미지 v2(10장)에 들어 있고 지금 Pod 는 v1. 10장 뒤 다시 본다
라벨 없는 Pod → 모델 서버	막힘(000) 또는 안 막힘(200, 기본 CNI)	**200 — 안 막힘.** 정책은 들어갔지만 기본 CNI 가 집행하지 않는다(예상한 쪽). 실제 클라우드와 다른 점으로 적는다

항목	예상	결과 (2026-10-12, 노트북)
상담 화면 Pod → 모델 서버	200	200

첫 실행은 Ingress 컨트롤러의 검증 웹훅이 뜨기 전에 매니페스트를 넣어 `connection refused` — 스크립트가 컨트롤러를 기다린 뒤 넣게 고쳤다.

---

09. 관문 — 로그인 · 요청 제한 · LiteLLM 게이트웨이

완료 목표 · Ingress 에 기본 인증(Basic Auth)과 초당 요청 제한을 더하고, 모델 서버 앞에 LiteLLM 게이트웨이를 두어 키 없는 요청을 401 로 막는다.

```
그림 09 | 사람은 로그인, 프로그램은 키
  브라우저 ──로그인(Basic Auth)──▶ Ingress(초당 5) ──▶ console ──Bearer 키──▶ litellm:4000 ──▶ sglang:30000
  실제 클라우드: Cognito/SSO + API Gateway(rate limit) + LLM 게이트웨이(키·사용량·대체 경로)
```

5주차의 게이트웨이 구성을 그대로 Pod 로 옮긴다. 다만 5주차의 **CPU 대체 엔진(llama.cpp)은 넣지 않는다** — 노트북 메모리가 모델 서버 하나로 이미 찼다(07장). 대체 경로의 동작은 5주차 측정(요청 손실 0, 60개)으로 갈음하고 여기선 키 검사와 통과만 본다.

실행

Ubuntu Bash | 09-001

```
bash deploy/scripts/09-gateway.sh agent agent-pass
kubectl -n lab set env deploy/console ONPREM_MODEL=gateway
ONPREM_MODEL_URL=http://litellm:4000/v1 && kubectl -n lab rollout status deploy/console
```

1행 `openssl passwd -apr1` 로 비밀번호 해시 → Secret `basic-auth` → Ingress 를 `09-ingress-auth.yaml` (인증 + `limit-rps: 5`)로 교체 → LiteLLM(ConfigMap 설정, 키는 04장의 Secret `model-keys`) → 측정: 인증 없이/있이, 40 요청 동시(503 수), 게이트웨이 틀린 키/맞는 키/실제 답. 결과 `results/cloud/09_gateway.txt` . 부록 A9. 2행 상담 화면이 모델 서버를 **게이트웨이 이름(`agent-llm-4b`)으로** 부르게 바꾼다(`defaults.MODELS["gateway"]` , 키는 `ONPREM_MODEL_KEY` 로 이미 들어 있다). 브라우저 `https://console.local/` 에 로그인 창이 뜨고, 관리자 화면의 모델이 `gateway` 다.

관찰 · 인증 없이 `401` , 계정으로 `200` . 40 요청 중 503 이 나온 수가 '초당 5 + 순간 5' 를 넘은 몫이다(네트워크 순서에 따라 25~35). 게이트웨이는 틀린 키 `401` , 맞는 키 `200` , 질문엔 `"content":"파란색"` 같은 답.

장애·복구 · 로그인 뒤 상담 화면의 SSE 가 끊기면 `proxy-buffering: off` 가 빠진 것. 503 이 0 이면 요청이 느려 초당 5 를 못 넘은 것 — `limit-rps` 를 2 로 내려 본다. LiteLLM 이 `CrashLoopBackOff` 면 ConfigMap 의 들여쓰기.

정리 · 없음. (요청 제한이 뒤 장의 측정을 방해하면 `kubectl apply -f deploy/manifests/08-ingress.yaml` 로 되돌린다 — 10장 이후 스크립트는 Ingress 를 거치지 않고 Service 로 직접 부르므로 영향 없다.)

항목	예상	결과 (2026-10-12, 노트북)
인증 없이 / 계정으로	401 / 200	**401 / 200**
40 요청 동시 → 503 수	25~35	**36**(통과 4) — 초당 5 + 순간 허용 5 보다 조금 더 엄격했다(40개가 1초 안에 들어가 '초당 5'만 적용된 셈)
게이트웨이 틀린 키 / 맞는 키	401 / 200	**400 / 200** — LiteLLM 은 틀린 키를 400 으로 거부한다(401 이 아님, 거부는 같음)
게이트웨이 경유 답	한 단어 답	`"content":"파랑"` (model agent-llm-4b, 생각 모드 끔) · 상담 화면 `ONPREM_MODEL=gateway` 전환 완료

첫 실행은 `auth-realm` 에 한글을 넣어 검증기가 거부(`invalid value`) — ASCII 로 바꿨다. 스크립트가 거기서 멈춰 LiteLLM 이 안 떴고, 그 사이 게이트웨이로 바꾼 상담 화면은 `/readyz`(엔진에 닿아야 함)를 못 넘겨 기다렸다 — LiteLLM 이 뜨자 저절로 Ready.

---

10. 데이터 — PostgreSQL + pgvector 하나로, 무상태 되찾기

완료 목표 · 05장에서 확인한 "승인 대기가 Pod 안에 있다"를 고친다. 대화·승인 대기(LangGraph 체크포인터)와 검색 색인(pgvector)을 PostgreSQL StatefulSet 하나에 두고, 색인은 Job 이 한 번 만들며, 상담 화면 3개 중 하나를 죽여도 승인이 이어지는지(200) 다시 잰다. DB Pod 를 죽여도 데이터가 남는지도 본다.

```
그림 10 | 상태는 전부 DB 에, Pod 는 비어 있다
  Job index-corpus ──(한 번)──▶ postgres-0 (pgvector: chunks_ops · chunks_cs · chunks_it)
  console ×3 ──체크포인트 읽기/쓰기──▶ postgres-0 (checkpoints)      ← 어느 Pod 가 받아도 같은 상담
              └ 검색: ONPREM_INDEX=reuse — 앱은 색인을 만들지 않고 읽기만
  PVC data ──▶ Pod 가 죽어도 남는다            실제 클라우드: RDS + pgvector 또는 별도 벡터 DB, 색인은 배치(Airflow)
```

2주차에 재 둔 pgvector 어댑터(`stores/pgvector.py`)와 7주차 웹의 체크포인터 자리를 그대로 쓴다. 바뀐 코드는 셋: ① 웹 `--store postgres`(`ONPREM_WEB_STORE`, `ONPREM_PG_DSN`) ② 창구 서버의 벡터 저장소를 `ONPREM_STORE` 로 고름(표 이름 `chunks_<창구>`) ③ `ONPREM_INDEX=reuse` 면 저장소에 이미 든 색인을 그대로 쓴다(문서 임베딩을 건너뛰어 앱 시작이 빠르다). 이미지에 `agent-pg` 묶음이 들어가므로 **이미지를 v2 로 다시 만든다**(03장 스크립트 그대로 — 코드 층만 바뀌어 1분).

실행

Ubuntu Bash | 10-001

```
bash deploy/scripts/03-build.sh v2
bash deploy/scripts/10-db.sh
```

1행 `agent-pg`(PostgreSQL 체크포인터·psycopg·pgvector)가 든 이미지 `onprem-agent:v2`. 설치 층이 바뀌므로 이번엔 캐시를 못 쓴다 — 1~2분. 2행 PostgreSQL(StatefulSet + PVC 2Gi, 이미지 `pgvector/pgvector:pg16`) → 색인 Job(창구 셋, 해시 임베더) → 표 확인 → 상담 화면을 v2 + postgres 저장 + pgvector 재사용으로 바꾸고 3개로 → **05장의 Pod 죽이기 그대로** → DB 의 체크포인트 수 → **DB Pod 를 죽이고** 다시 세어 데이터가 남는지. 결과 `results/cloud/10_db.txt`. 부록 A10.

예상 · 05장과 같은 시험에서 이번엔 `HTTP 200` 과 `"done": "잠금 해제를 요청했습니다"` — 승인이 다른 Pod 에서 이어진다. DB Pod 를 지워도 `checkpoints`·`chunks_it` 행 수가 같다. 색인 Job 은 해시 임베더라 수 초.

관찰 — 임베더를 bge-m3 로 · 노트북 기본값(bge-m3)으로 색인하려면 Job 의 `ONPREM_EMBEDDER` 를 `{"kind":"sentence-transformers","model":"BAAI/bge-m3"}` 로 바꿔야 하는데 이미지에 torch 가 없다(03장 결정). 이 워크북에선 해시 임베더로 흐름만 보고, bge-m3 색인은 **호스트에서** 돌려 같은 DB 에 넣는 길을 적어 둔다: `kubectl -n lab port-forward svc/postgres 5432:5432 &` 뒤 `ONPREM_STORE='{"kind":"pgvector","dsn":"postgresql://agent:agent@localhost:5432/agent"}' ONPREM_EMBEDDER='{"kind":"sentence-transformers","model":"BAAI/bge-m3"}' python -c "from onprem_agent.agent.tool_server import build_search; build_search(['examples/desks/it/data'])"`. 그러면 Pod 쪽은 질문 임베딩만 하면 되는데 — 그것도 bge-m3 가 필요하다. 즉 **검색 임베더는 결국 앱 이미지에 들어가야** 하고(이미지 3GB), 그걸 피하려면 임베딩을 별도 서비스(임베딩 서버)로 뺀다. 이 결정은 9주차로 넘기고 여기 적는다.

장애·복구 · 색인 Job 이 `relation … does not exist` 면 DB 가 아직 안 떠서 — `rollout status sts/postgres` 뒤 Job 을 지우고 다시(`kubectl -n lab delete job index-corpus`). 상담 화면이 `/readyz` 503 이면 `ONPREM_PG_DSN` 이 틀린 것(로그에 `connection refused`).

정리 · 상담 화면 1개로(`scale --replicas=1`). PostgreSQL 은 11·12장이 쓴다.

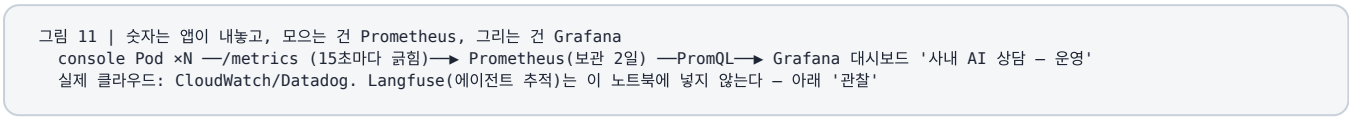
항목	예상	결과 (2026-10-12, 노트북)
색인 Job 시간 · chunks_it 행 수	수 초 · 수십	**7초** · 표 3개(chunks_ops·chunks_cs·chunks_it), IT **12행**. 첫 Job Pod 는 DB 의 DNS 가 생기기 전에 떠서 `failed to resolve host 'postgres'` 로 실패, 둘째가 성공(스크립트가 DB 준비 뒤 Job 을 만들게 고침)
승인 대기 중 Pod 삭제 → 같은 상담 번호 승인	**200** (05 장은 409)	**200** — 질문은 `6rxnb` 가 받아 담당자 확인에서 멈춤 → 그 Pod 삭제 → 승인은 `2tc27` 이 받아 이어서 실행, `[RQ-0001] 계정 잠금 해제 요청 접수됨: E1063`. 체크포인트 7개 · 상담 1건이 DB 에
DB Pod 삭제 → 체크포인트·색인 행 수	같음	**7 / 12 그대로**(PVC)
이미지 v2 크기	v1 + 수십 MB	**214MB**(+6MB), 빌드 39초

답 글은 "요청이 처리되었습니다. [RQ-0001] … 하지만 최종 승인은 관리자에게 요청해야 합니다" — 실행은 됐는데 모델이 말끝을 흐렸다(거짓 보고의 반대 방향: 한 일을 덜 했다고 말함). 채점기는 성공으로 보지만 읽어 두는 종류의 답이다.

---

11. 관측 — /metrics · Prometheus · Grafana

완료 목표 · 상담 화면이 숫자(질문 수 · 처리 시간 분포 · 확인 대기 · 승인 · 오류)를 `/metrics` 로 내놓고, Prometheus 가 15초마다 긁어 가며, Grafana 대시보드에 '초당 질문 수 · p95 처리 시간 · 확인 대기 수 · Pod 수'가 그려진다. 부하를 걸어 p95 가 숫자로 읽히는지 본다.



지표는 **의존성 없이** 앱이 Prometheus 글자 형식으로 직접 쓴다(`web/app.py` 의 `/metrics`): `agent_chat_total` (질문 수), `agent_chat_status_total{status}` (답함·확인 대기·오류), `agent_approve_total` , `agent_turn_seconds` (히스토그램 — 0.5초부터 120초까지 9칸), `http_requests_total{path,code}` . 이걸 담은 이미지가 **v3** 다. Prometheus 는 `prometheus.io/scrape: "true"` 주석이 붙은 Pod 를 자동으로 긁는다(04 매니페스트에 주석 추가).

실행

Ubuntu Bash | 11-001

```
1 bash deploy/scripts/03-build.sh v3
2 bash deploy/scripts/11-observe.sh 60 4
3 kubectl -n lab port-forward svc/grafana 3000:80
```

1행 `/metrics` 가 든 이미지 v3(코드 층만 바뀌어 1분). 2행 helm 으로 Prometheus(서버만, 알림·익스포터 끔, 256Mi~768Mi)와 Grafana(128~384Mi, 데이터 원본과 대시보드를 파일로 심음) → 상담 화면에 스크레이프 주석 + v3 → Prometheus 대상 목록에 console 이 `up` 인지 → 부하 60초(동시 4, IT 창구 규정 질문) → 90초 뒤 Prometheus API 로 초당 질문 수 · p95 · 누적을 읽는다. 결과 `results/cloud/11_observe.txt` . 부록 A11. **지금 상담 화면은 게이트웨이 → 실제 모델**이라 질문 하나가 10~60초 걸리고 p95 도 그 크기다(규칙 모델이면 1초 아래). 어느 모델인지 스크립트가 같이 찍는다. 3행 브라우저 `http://127.0.0.1:3000` (admin / lab-admin) → 대시보드 '사내 AI 상담 — 운영'. 부하가 끝난 뒤라 그래프의 꼬리만 보일 수 있다 — 2행을 한 번 더 돌리며 보면 움직인다.

관찰 · Grafana 의 p95 선이 Prometheus API 의 숫자와 같다(같은 PromQL). '확인 대기'는 규정 질문 부하에서는 0 이고, 10장의 잠금 해제 상담을 한 번 하면 1 이 된다. **Langfuse(에이전트 추적)는 넣지 않았다** — v3 부터 ClickHouse·Redis·PostgreSQL 셋이 더 필요해 노트북 메모리(07장 예산) 밖이다. 대신 이 프로젝트의 추적은 이미 있다: 전체 시험의 `results/*.json` 이 과업마다 호출·안전 기능 발동을 남기고(7주차), 상담 화면의 '최근 상담'이 실시간 쪽이다. 실제 클라우드라면 Langfuse 를 별도 노드에 두고 LangGraph 콜백 하나로 붙인다(6주차 조사).

장애·복구 · 대상 목록에 console 이 없으면 주석이 안 붙은 것(`kubectl -n lab get pod -l app=console -o jsonpath='{.items[0].metadata.annotations}'`). `up` 인데 지표가 `없음` 이면 아직 15초 주기가 안 지났거나 이미지가 v2(→ `/metrics` 404)다. Grafana 대시보드가 비어 있으면 ConfigMap 라벨이 아니라 `dashboardsConfigMaps` 값 이름(`lab`)이 맞는지.

정리 · 11장 서비스는 12장·13장이 쓰니 둔다. 내리려면 `helm -n lab uninstall prometheus grafana` .

항목	예상	결과 (2026-10-12, 노트북)
Prometheus 대상에 console (`up`)	1개 이상	**1, up**

항목	예상	결과 (2026-10-12, 노트북)
부하 중 초당 질문 수 (Pod 별)	실제 모델: 0.1~0.3 · 규칙: 수십	**0.267**(게이트웨이 → Qwen3-4B(FP8), 생각 모드)
처리 시간 p95 (초)	실제 모델: 10~60 · 규칙: <1	**18.6초**
질문 누적 / 확인 대기 / 오류	n / 0 / 0	20 / (측정기 따옴표 실수로 못 읽음 — 고쳐 둠, 다음 실행에서)
Grafana 대시보드	네 칸 모두 숫자	(캡처)

이미지 v3 214MB(코드 층 +4KB), 빌드는 캐시. Prometheus·Grafana 설치 각 1분 안쪽(grafana 차트는 'deprecated' 경고 — 동작엔 문제 없음, 13장 `cloud-up` 에서도 같은 차트).

---

12. CI/CD — Jenkins 를 클러스터에, 노트북 GPU 를 일꾼으로

완료 목표 · Jenkins 제어부를 helm 으로 클러스터에 올리고, **노트북을 인바운드 일꾼(`laptop`, 라벨 gpu)** 으로 붙인다. 저장소에 올리면(5분마다 poll) 단위·모듈 시험이 돌고, 매일 밤 2시에는 전체 시험(1시간, GPU)이 돈다 — 시작 전에 클러스터의 모델 서버·상담 화면을 내려 GPU 를 비우고 끝나면 올린다. 빨간불이면 빌드가 실패로 남는다.

```
그림 12 | 제어부는 안, 일꾼은 밖
  GitHub ◀─poll 5분─ Jenkins(클러스터, 768Mi~1.5Gi, PVC) ──웹소켓(일꾼이 먼저 연결)──▶ 노트북 일꾼 'laptop' (GPU, .venv)
                     │ 올릴 때: 단위 → 모듈                                          │ 밤 2시: 12-quiet.sh down → make test-full → up
                     └ results/*.md 보관(archiveArtifacts)                           └ 빨간불 1건이면 error → 빌드 실패
  실제 클라우드: GitHub Actions/ArgoCD + GPU 러너 풀. 여기선 '일꾼이 GPU 를 가진 쪽에 있다'는 모양이 핵심
```

7주차의 시험 세 단계가 그대로 파이프라인이다(`Jenkinsfile`): `make test-unit` → `make test-module` → (타이머로 시작됐을 때만) `make test-full CONFIG=configs/agent_laptop_w7_split.yaml` 과 빨간불 검사. 일꾼은 노트북의 `.venv` 파이썬을 쓰되 코드는 Jenkins 가 내려받은 작업 공간의 것(`PYTHONPATH=$WORKSPACE/src`)이다 — 편집 설치가 `~/onprem-agent/src` 를 가리키므로 이렇게 덮어야 "올린 코드"를 시험한다.

실행

Ubuntu Bash | 12-001

```
1 bash deploy/scripts/12-jenkins.sh
2 kubectl -n lab port-forward svc/jenkins 8081:8080
3 sudo apt install -y openjdk-17-jre-headless
4 bash deploy/scripts/12-jenkins.sh agent
```

1행 helm 으로 Jenkins(JCasC 설정: 일꾼 `laptop` 정의, 저장소를 보는 파이프라인 잡 `onprem-agent`, 제어부 실행기 0). Ready 까지의 시간을 적는다(이미지 받기 포함 3~6분). 부록 A12. 2행 **다른 창**에서 제어부를 노트북 8081 로 잇는다(켜 둔다). 브라우저 `http://127.0.0.1:8081` (admin / lab-admin). 3행 일꾼은 자바로 돈다 — 없으면 설치. 4행 **또 다른 창**에서 일꾼을 붙인다: 제어부에서 `agent.jar` 와 `laptop` 의 비밀을 받아 웹소켓으로 연결(켜 둔다). Jenkins 화면의 '노드'에 `laptop` 이 온라인이 된다.

관찰 · 잡 `onprem-agent` 를 열어 **지금 빌드**를 누른다(또는 커밋을 하나 올리고 5분 기다린다) → 단위 → 모듈 두 단계가 노트북에서 돈다(모듈 시험 약 1분). 이 시간을 적는다. **밤 2시 전체 시험은 내일 아침** 빌드 기록과 `results/*_gate.md`(보관된 산출물)로 확인한다 — 12-quiet.sh 가 모델 서버를 내렸다 올린 로그도 거기 있다.

장애·복구 · 일꾼이 `offline` 이면 4행의 창이 닫혔거나 2행의 port-forward 가 끊긴 것(둘 다 켜 둬야 한다). 빌드가 `make: command not found` 면 일꾼 셸의 PATH — `.venv` 는 절대 경로라 괜찮고 `make` 는 WSL 에 있다. 밤 시험이 `CUDA out of memory` 면 12-quiet.sh 가 모델 서버 Pod 를 못 내린 것 — `kubectl -n lab get pods -l app=sglang`.

정리 · 두 창(port-forward · 일꾼)은 밤 시험을 보려면 켜 둔다. 끄면 밤 시험은 '일꾼 없음'으로 대기한다.

항목	예상	결과 (2026-10-12, 노트북)
Jenkins Ready 까지	3~6 분	설정 오류 셋을 고치며 약 30분 — ① 플러그인 이름 `timestamps` → `timestamper` (init 반복 실패) ② JCasC `numExecutors` 가 차트 기본 설정과 충돌 → 차트 값으로 ③ 일꾼 `remoteFS` 가 없는 경로(`AccessDenied`, 빌드 #1 3.8초 실패) → 실제 폴더. 마지막 `helm upgrade` 뒤 82초에 `2/2 Running`

항목	예상	결과 (2026-10-12, 노트북)
일꾼 `laptop` 온라인	됨	**됨** — 웹소켓 `Connected`, 응답 286ms. 빌드 중 9초 끊겼다 다시 붙음(`laptop is back online`)
올릴 때 빌드(단위+모듈) 시간	1~2분	**#2 SUCCESS, 약 70초** — 단위 39 통과(12초), 모듈 5 통과 + 규칙 모델 평가 셋(기존 26 · 보호 20 · 경계 6 · 확인용 26 · IT 22/23), 빨간불 검사 초록불 ×3, 밤 단계는 `when` 으로 건너뜀, 산출물 보관
밤 전체 시험(다음 날 확인)	1시간, 초록불	(2026-10-13 아침에)

로그의 `FileNotFoundError: ''` 는 Makefile 이 `CONFIG` 없이도 결과 이름을 계산하려던 것 — 조건부로 고침(빌드엔 영향 없었음).

---

13. IaC — `make cloud-up` 한 줄, 처음부터 다시 올리는 시간

완료 목표 · 01~11장을 스크립트 하나(`deploy/scripts/cloud-up.sh`)로 순서대로, **다시 돌려도 되게(멱등)** 묶고, 클러스터를 지운 뒤 처음부터 올리는 데 걸리는 시간을 잰다. 이미지 태그 `latest` 는 digest 로 고정한다(C11 의 교훈).

```
그림 13 | 매니페스트·값 파일이 곧 설계도
  make cloud-up ──▶ cloud-up.sh: 01 클러스터 → 03 이미지 v3 → 04 설정·화면 → 07 모델 서버(digest 고정) → 10 DB·색인 → 08·09 엣지·관문 → 11 관측 → 확
  make cloud-down ──▶ minikube stop (내용 유지)        make cloud-destroy ──▶ minikube delete (전부 삭제)
  실제 클라우드: Terraform/Helm 차트. 여기선 kubectl apply + helm upgrade --install 의 멱등성이 그 역할
```

실행

Ubuntu Bash | 13-001

```
   1  make cloud-up                                  # 지금 상태에서 — 전부 이미 있으니 '확인'만
하고 몇 분 안에 끝나야 한다
   2  make cloud-destroy && make cloud-up            # 처음부터 — 이미지 36GB 받기·가중치·컴파일
캐시가 다 사라지므로 길다. 시간을 적는다
```

1행 멱등성 확인: 있는 것은 건너뛰고(`image ls`, `get job`, `apply` 의 unchanged) 끝에 `https 200` 과 Pod 목록. 걸린 시간이 '다시 돌리기 비용'이다. 2행 **선택** — 사라지는 것: SGLang 이미지(36GB, 다시 받기), 가중치 PVC(4.4GB), JIT 캐시(07장 첫 시작의 긴 컴파일을 다시), DB. 인터넷과 컴파일 때문에 1시간을 넘길 수 있다. 할 때는 다른 일을 하면서. 끝나면 `results/cloud/13_cloud_up.txt` 의 `[초] 단계` 줄이 단계별 시간표다.

관찰 · 1행과 2행의 차이가 '캐시의 값'이다 — 이미지·가중치·컴파일 캐시 셋. 실제 클라우드에서 이 셋을 레지스트리·객체 저장소·베이스 이미지로 미리 두는 이유.

장애·복구 · 2행에서 07 단계가 길면 C11 의 첫 시작(컴파일)이다 — `kubectl -n lab logs -l app=sglang -f` 로 진행을 본다. Jenkins 는 기본 건너뜀(`WITH_JENKINS=1 make cloud-up` 으로 포함) — 일꾼은 사람이 붙여야 해서.

항목	예상	결과 (2026-10-12, 노트북)
1행: 있는 상태에서 `make cloud-up`	2~5분	**1분 30초** — 01~11 단계 19초(전부 있음·unchanged), 기다림 71초(모델 서버가 digest 고정으로 재시작 → 73초 Ready, 캐시 덕). 끝 `https 200 Pod console-…-bqrpc`, Pod 10개 Running/Completed
2행: 처음부터 (선택)	40~90분	하지 않음 — 36GB 이미지와 컴파일 캐시(C11)를 잃는다. 하면 07 단계가 다시 수십 분
끝의 `https 200`	200	**200**

가중치·색인 Job 은 `ttlSecondsAfterFinished` 로 이미 지워져 있어 다시 만들어졌다(가중치는 캐시에 있어 수초, 색인 7초) — 멱등이 '다시 안 함'이 아니라 '다시 해도 같은 결과'라는 뜻임을 보여 준다.

마치며 — 이 워크북에서 잰 것

층	잰 것	결과
이미지	크기 · 빌드 · 캐시	208MB → 214MB(v2), 43초 → 1초(캐시)
로드 밸런서	Pod 별 요청 수	9·8·10 (측정기 결함 D45 를 잡은 뒤 30/30)
무상태	승인 대기 중 Pod 삭제	409(SQLite in Pod) → **200**(PostgreSQL 체크포인터)
HPA	1→4→1	지표 60초 지연, 늘기 즉시, 줄기 106초
GPU 모델 서버	첫 시작 · 둘째 시작 · 메모리	2시간 40분(C11) · **58초** · 7.9GB 안정 — 00장 예산 틀림
엣지	308/200/304 · NetworkPolicy	됨 · **기본 CNI 는 미집행**
관문	401/200 · 40 동시 · 게이트웨이	됨 · 36 차단 · 400/200, "파랑"
DB	DB Pod 삭제 뒤 데이터	7/12 유지
관측	p95 · 초당 질문	18.6초 · 0.27/s (실제 모델)
CI/CD	올릴 때 빌드	Jenkins + 노트북 일꾼, 70초 SUCCESS, 빨간불 검사 초록불 ×3 · 밤 시험은 다음 날
IaC	`make cloud-up` 다시 돌리기	1분 30초, https 200

세 가지가 감과 달랐다 — **메모리 예산**(모델 서버 4~5GB 가 아니라 7.9GB, 첫 시작은 12GB 넘게), **무상태는 저절로 되지 않는다**(Pod 를 늘릴 수 있어도 상태가 안에 있으면 늘린 게 아니다), **측정기부터 의심**(30회 중 3회 빠진 건 요청이 아니라 `kubectl run -i` 였다). 문제해결 이력 C9·C10·C11, D45.

부록 — 스크립트 한 줄씩

A0. `deploy/scripts/install-minikube.sh` · `command -v X || {…}`: X 가 없을 때만 설치. kubectl 은 `dl.k8s.io` 의 최신 안정판, minikube 는 공식 릴리스, helm 은 공식 설치 스크립트. 끝에 세 버전을 찍는다.

A1. `deploy/scripts/probe-gpu.sh` · `set -u` 만 켜고 `-e` 는 끈다 — 실패해도 끝까지 기록해야 판정이 되니까. 호스트 `nvidia-smi` 와 Docker 런타임 → minikube 상태 → 기다리기 세 가지(새 네임스페이스의 `default` 서비스 계정 · 장치 플러그인 DaemonSet 의 rollout · 노드의 `nvidia.com/gpu` 광고) → CUDA 기본 이미지 Pod 를 `--overrides` 로 `nvidia.com/gpu: 1` 을 요구하며 띄워 `nvidia-smi -L` → Pod 의 Events 와 노드 광고 수. 전부 `tee -a` 로 `results/cloud/01_gpu_probe.txt` 에 쌓인다(C9 의 교훈이 기다리기 세 줄).

A3. `deploy/scripts/03-build.sh` · `docker build -q` (조용히, 이미지 id 만) 앞뒤로 `date +%s` 를 찍어 초를 센다 → `docker image inspect` 로 크기와 층 수 → `docker history` 로 층별 크기 → 호스트에서 `docker run -d --rm -p 18080:8080` 으로 띄워 30초 안에 `/healthz` 가 200 이 되는지 → 멈춤 → `minikube image load` 시간 → 노드 창고 목록. `tee` 로 `results/cloud/03_image.txt`.

A4. `deploy/scripts/04-config.sh` · `kubectl create configmap --from-file=desk.yaml=<파일> --dry-run=client -o yaml | kubectl apply -f -`: '만들기'를 '있으면 덮기'로 바꾸는 상투구. Secret 도 같은 꼴, 값은 `${GATEWAY_KEY:-sk-local-dev}` (환경 변수가 있으면 그것, 없으면 개발용) — 키를 Git 에 두지 않는다.

A6. `deploy/scripts/06-load.sh` · 부하 Pod 하나를 `kubectl run load … curl` 로 띄운다: 끝 시각까지 `동시 수` 만큼 `curl -X POST /api/chat … &` 를 띄우고 `wait` (한 묶음), 반복하며 요청 수를 센다. 호스트 쪽 루프는 (부하 초 + 90초)/10 번 돌며 `kubectl get hpa -o jsonpath` 로 현재 사용률/목표, `get deploy` 로 목표 Pod 수와 준비된 Pod 수를 한 줄씩 찍는다. 끝에 `top pod` 와 HPA 의 Events.

A8. `deploy/scripts/08-edge.sh` · `openssl req -x509` 로 자체 서명 인증서(한 번만, `var/`) → `create secret tls` → Ingress·NetworkPolicy apply → 컨트롤러 준비 대기 → `curl --resolve console.local:443:<minikube ip>` 로 DNS 없이 host 를 맞춰 다섯 가지를 찍는다(308 · 200/인증서 · ETag 304 · 라벨 없는 Pod 의 curl · console Pod 의 urllib).

A9. `deploy/scripts/09-gateway.sh` · `openssl passwd -apr1` 해시로 `var/auth` (htpasswd 꼴) → Secret `basic-auth` → 인증·제한 Ingress 로 교체 → LiteLLM apply → curl 401/200 → 40개 curl 을 `&` 로 동시에 띄워 코드를 `sort | uniq -c` → 1회용 Pod 에서 게이트웨이에 틀린 키·맞는 키·실제 질문.

A10. `deploy/scripts/10-db.sh` · PostgreSQL apply·대기 → Job 완료 대기와 시간 → `psql \dt` ·행 수 → 상담 화면 `set image v2` + `set env` (postgres 저장·pgvector·reuse) → 3개로 → `05-kill.sh` 재사용 → `checkpoints` 집계 → `delete pod postgres-0` 뒤 다시 집계.

A11. `deploy/scripts/11-observe.sh` · helm repo 추가 → 대시보드 ConfigMap → Prometheus·Grafana `upgrade --install --wait` → 상담 화면에 스크레이프 주석 `patch` (04 매니페스트를 `apply` 하면 10장의 env·이미지가 되돌아가므로) + 이미지 v3 → Prometheus `/api/v1/targets` 에서 console 대상 → 부하 Pod → `/api/v1/query` 로 PromQL 셋.

A12. `deploy/scripts/12-jenkins.sh` · (기본) helm 으로 Jenkins 올리고 Ready 시간. (`agent`) `agent.jar` 를 제어부에서 받고, `/computer/laptop/jenkins-agent.jnlp` 에서 비밀(64자리 hex)을 뽑아 `java -jar agent.jar -webSocket` 으로 붙는다. `12-quiet.sh down|up` 은 sglang·console·litellm 을 0/1 로.

A13. `deploy/scripts/cloud-up.sh` · 단계마다 `[경과초] 이름` 을 찍는다. 멱등 장치: `minikube status` 로 있으면 start 만, `image ls | grep` 으로 있으면 빌드 안 함, Job 은 `get job` 으로 있으면 다시 안 만듦, Secret 은 `--dry-run=client | apply` , helm 은 `upgrade --install` . 끝에 Ingress 를 통해 `https 200` 확인.

A5. **`deploy/scripts/05-lb.sh`** · **`05-kill.sh`** · 둘 다 `kubectl run … --image=curlimages/curl --rm -i --quiet` 로 클러스터 안에 1회용 curl Pod 를 띄운다. lb: `curl -w '%header{X-Served-By}'` 로 답한 Pod 이름만 N 줄 찍고 `sort | uniq -c`. kill: ① `/api/chat` 에 `-N`(버퍼 없이)으로 SSE 를 받다가 `approval` 사건이 보이면 끊고(`grep -m1`), `-D` 로 받은 머리말에서 답한 Pod 이름을 읽는다 ② 그 Pod 를 `delete --wait=false` ③ 두 번째 curl Pod 가 같은 상담 번호로 `/api/approve` 를 보내고 HTTP 코드와 답한 Pod 를 찍는다.